SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: **Artificial Intelligence** **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:50

Annexure A

<u>Analytical Problem Solving</u>

Code of Conduct:

I M·SHAKTHI 192312538 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

25+25+25+13+8

= 96

Student 17: M SHAKTHI | Register Number: 192312538

1. A ride-hailing company operates a network of pickup points connected by roads with travel costs (time + fuel). Using the Uniform Cost Search (UCS) algorithm, determine the least-cost path from a starting point S to a destination G. Describe the frontier and explored set at each step for a graph of your choice with at least 6 nodes.

2. Compare Bidirectional Search with standard Breadth-First Search for finding a path between a start node S and goal node G in a graph with 10 nodes. Illustrate with an example why bidirectional search can significantly reduce time complexity.

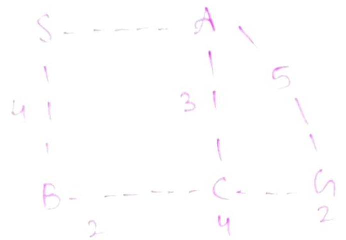
3. A vacuum-cleaner robot operates in a two-room environment (Room A and Room B), each of which can be clean or dirty. Define the PEAS (Performance measure, Environment, Actuators, Sensors) description for this agent, and characterize its task environment along the dimensions of observability, determinism, episodicity, and number of agents.

Uniform Cost Search (UCS) for a ride-Hailing Network:

Introduction: UCS is an uninformed search algorithm that always expands the node with the lowest total path cost from the starting node. In a ride-hailing system, the cost can represent the combined travel time and fuel cost. UCS is useful for finding the least-cost path from a pickup point S to a destination G.

Example graph:

Consider the following road network with 7 nodes:



additional connection:

A → D = 6

D → G = 2

C → G = 2

The edges & costs are:

S → A - 2	A → C - 3	B → C - 2	C → D - 4
S → B - 4	A → D - 6	B → G - 2	D → G - 2

UCS steps:

Frontier : Nodes discovered but not yet expanded.

Explored set : Nodes that have already been expanded.

Initially, Frontier : {S(0)}

Explored : { }

Step 1: Expand S

from S: S → A = 2

S → B = 4

∴ Frontier : {A(2), B(4)}

Explored : {S}

UCS chooses A because it has the lowest cost.

Step 2: Expand A

from A: A → C = 2+3 = 5

A → D = 2+6 = 8

∴ Frontier: {B(4), C(5), D(8)}

Explored: {S, A}

UCS chooses B because its cost is 4.

Step 3: Expand B

from B: B → C = 4+2 = 6

But C is already in the frontier with cost 5. ∴, the cheaper path through A is retained.

Frontier: {C(5), D(8)}

Explored: {S, A, B}

UCS chooses C

Step 4: Expand C

from C: C → G = 5+2 = 7

C → D = 5+4 = 9

The path to G has cost 7.

Frontier: {G(7), D(8)}

explored: {S, A, B, C}

UCS chooses G because it has the lowest cost.

Step 5: G is the goal node, so the search stops.

The least cost path is: S → A → C → G ⇒ 2+3+2 = 7

<u>Conclusion</u>: UCS guarantees an optimal solution when all edge costs are non-negative. So, it is suitable for ride-hailing applications where different roads have different travel costs.

Bidirectional Search vs Breadth - First Search :

Introduction : Breadth first search starts from the source node S and explores the graph level by level until it reaches the goal G.

Bidirectional Search improves this approach by searching simultaneously.

* Forward from the start node S.
* Backward from the goal node G.

Example with 10 nodes :

Start = S

Goal = G

For illustration, consider a branching factor of approximately 2 and a path b/w S & G of depth 4.

Standard BFS : BFS starts at S and explores:

Level 0 : S

Level 1 : A, B

Level 2 : C, D, E

Level 3 : F, G, H

BFS continues expanding nodes until it reaches G.

The important point is that BFS searches only from starting point.

Bidirectional Search :

It works from both ends.

S → → → ← ← ← G

The forward search starts from S while the Backward search starts from G.

For a path of depth 4:

Forward : S → Level 1 → Level 2

Backward : G → Level 1 → Level 2

Complexity comparison :

If ; Branching factor = b

solution depth = d

For BFS, the approx time complexity is O(b^d)

For BDS, each search explores approx half the depth :

O(b^(d/2)) + O(b^(d/2))

∴ the search can be significantly smaller.

for ex : if b = 2, d = 6,

BFS ⇒ 2^6 = 64 nodes

BDS ⇒ $2^3 + 2^3$ = 16 nodes

Thus, the no. of nodes explored can be greatly reduced.

Comparison :

Feature	BDS	BFS
Search direction	Start → middle → Goal	Start → Goal
no. of searches	Two	one
time complexity	O(b^(d/2))	O(b^d)
Memory	Generally lower per search	High
main advantage	Faster for suitable problems	Simple implementation

Conclusion : BDS can significantly reduce the search space because it searches from both ends and meets near middle. It is especially effective when the start & goal nodes are known & the graph supports reverse searching.

PEAS Description for a Vacuum - cleaner Robot

Introduction : A vacuum cleaner robot is an intelligent agent that observes the condition of rooms and performs actions to keep them clean.

Consider an environment containing 2 rooms ie A, B.
Each room can either be clean / dirty.

PEAS Description :

P - performance measure
E - environment
A - actuators
S - sensors

P - Performance Measure :

The performance of the vacuum robot can be measured using : Keeping both rooms clean
cleaning dirty rooms efficiently
minimizing unnecessary movement
minimizing energy consumption
completing cleaning in minimum time.

E - Environment :

The environment consists of room A, B, vacuum cleaner robot, dirt in either room, connection / path b/w the rooms.

clean / Dirty : So, A clean - B clean , A clean - B dirty , A dirty - B clean , A dirty - B dirty.

A - Actuators :

The robot uses actuators to perform actions :
Move left , move right , suck / clean .

S - Sensors :

The robot can use sensors to detect :

current room / location,

whether the room is clean / dirty,

obstacles / boundaries,

Dirt .

Task environment characteristics :

1) observability - Fully observable :

The environment is considered fully observable if the robot can determine the relevant state of both rooms.

2) Determinism - Deterministic

3) Episodicity - Sequential

4) number of agents - single agent

PEAS component	Description
P	clean rooms, minimize time, movement energy
E	Two rooms, Room A & B with clean / dirty states
A	move left / move right, suck / clean
S	Location sensor, dirt sensor, obstacle sensor

Conclusion :

The vacuum cleaner robot is a simple intelligent agent that senses the environment, decides whether cleaning is required . suitable actions.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 [illegible]	[illegible]
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

25+25+25+13+8
= 96 Hp